

Score-Based Generative Modeling with SDEs: Theory and Implementation in TorchDiff

Loghman Samani
samanihqman91@gmail.com
Developer of TorchDiff

June 13, 2025

Abstract

This article explores score-based generative modeling through stochastic differential equations (SDEs), a powerful framework for transforming complex data distributions into noise and back using forward and reverse diffusion processes. We introduce key concepts, including the Variance Exploding (VE), Variance Preserving (VP), and sub-VP SDEs, as well as the deterministic Probability Flow ODE, which enables exact likelihood computation. Through detailed explanations and mathematical formulations, we connect these methods to discrete diffusion models like SMLD and DDPM. Additionally, we demonstrate how to implement SDE-based models using TorchDiff, a PyTorch-based library, with practical code examples for training and sampling on the MNIST dataset. This article serves as a comprehensive guide for researchers and practitioners interested in advanced generative modeling and its implementation.

1 Introduction to Stochastic Differential Equations in Diffusion Models

Stochastic Differential Equations (SDEs) form a powerful mathematical framework for modeling systems that evolve under both deterministic and stochastic influences. These equations find applications in diverse fields, including physics, finance, and machine learning, particularly in generative modeling. An SDE describes the evolution of a system state x_t over time t and is generally expressed as:

$$dx_t = f(x_t, t) dt + g(t) dW_t \quad (1)$$

Here, x_t represents the state of the system at time t , evolving from an initial data distribution p_0 to a prior distribution p_T over the interval $[0, T]$. The equation comprises two primary components:

- **Drift term** ($f(x_t, t) : \mathbb{R}^d \times [0, T] \rightarrow \mathbb{R}^d$): This deterministic component governs the systematic evolution of the system. It defines the direction and magnitude of the state's

change, such as the continuous transformation of a chemical system driven by a series of reactions over time.

- **Diffusion term** ($g(t) dW_t$, where $g(t) : [0, T] \rightarrow \mathbb{R}$ is the diffusion coefficient and dW_t is the increment of a Wiener process): This stochastic component introduces random fluctuations, typically modeled as Gaussian noise with mean 0 and variance dt . The function $g(t)$ modulates the noise magnitude, which may vary over time, allowing the noise level to increase or decrease as the process evolves from $t = 0$ to $t = T$. While the Wiener process (Brownian motion) is common in diffusion models, other stochastic processes, such as Lévy processes, can also be employed.

In the context of diffusion models, SDEs provide a principled approach to transforming complex data distributions (e.g., the distribution of an image dataset) into a simple, tractable distribution (e.g., a standard normal distribution) through a forward diffusion process. This process incrementally adds noise to the data over a series of timesteps. The reverse process, governed by a reverse-time SDE, reconstructs the original data distribution by gradually denoising samples drawn from the prior distribution. For a detailed introduction to diffusion models, see my previous article, *TorchDiff: A PyTorch Library for Denoising Diffusion Probabilistic Models and Beyond*.

The reverse process relies on the *score*, defined as the gradient of the log-probability density of the noisy data distribution at time t , denoted $\nabla_x \log p_t(x)$. This score guides the denoising process by indicating how to adjust noisy samples to make them more likely under the current distribution. Crucially, the reverse-time SDE depends only on this score, which can be approximated using a neural network trained via *score matching*. This approach avoids the need to compute the intractable probability density $p_t(x)$ directly, making the reverse process computationally efficient. By iteratively applying the learned score function, diffusion models can generate high-quality samples from the target data distribution.

2 Foundations of Diffusion Models

Diffusion models, such as Score Matching with Langevin Dynamics (SMLD) and Denoising Diffusion Probabilistic Models (DDPM), are powerful generative modeling frameworks that perturb data with noise and learn to reverse this process to generate high-quality samples. These methods form the foundation for score-based generative modeling with stochastic differential equations (SDEs), which unifies and generalizes their approaches. Below, we introduce SMLD and DDPM before discussing how SDEs provide a continuous-time framework for diffusion modeling.

2.1 Score Matching with Langevin Dynamics (SMLD)

Score Matching with Langevin Dynamics (SMLD) generates samples by estimating the score, defined as the gradient of the log-probability density, $\nabla_x \log p_t(x)$, of noisy data distributions at various noise scales. The process begins with a forward diffusion phase, where training data is progressively corrupted by adding Gaussian noise with increasing variance, creating a sequence of noisy distributions $p_t(x)$, with t indexing the noise level.

A neural network is trained to approximate the score function:

$$s_\theta(x, \sigma) \approx \nabla_x \log p_\sigma(x), \quad (2)$$

where $s_\theta(x, \sigma)$ is the neural network’s prediction of the score for data x at noise level σ , and $p_\sigma(x)$ is the noisy data distribution. The training objective, known as score matching, minimizes the difference between the predicted and true scores, typically using a loss function such as the Fisher divergence.

During the sampling phase, SMLD uses Langevin dynamics to generate samples from the learned score function. Starting from a noisy sample (e.g., drawn from a Gaussian distribution), Langevin dynamics iteratively updates the sample according to:

$$x_{t+1} = x_t + \epsilon \cdot s_\theta(x_t, \sigma_t) + \sqrt{2\epsilon} \cdot z, \quad (3)$$

where ϵ is a step size, $z \sim \mathcal{N}(0, I)$ is Gaussian noise, and σ_t corresponds to a decreasing sequence of noise scales. Over multiple iterations, this process moves samples toward regions of higher probability, approximating the target data distribution $p_0(x)$.

2.2 Denoising Diffusion Probabilistic Models (DDPM)

Denoising Diffusion Probabilistic Models (DDPM) learn to generate samples by reversing a forward diffusion process that gradually corrupts data with noise. In the forward process, data is perturbed over T timesteps according to a predefined noise schedule, forming a Markov chain of noisy distributions:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I), \quad (4)$$

where $\beta_t \in (0, 1)$ controls the noise variance at timestep t . This process transforms the original data x_0 into nearly pure noise $x_T \sim \mathcal{N}(0, I)$ over T steps.

In the reverse process, a neural network is trained to approximate the reverse Markov chain $p_\theta(x_{t-1}|x_t)$, which predicts a less noisy sample x_{t-1} given a noisy sample x_t :

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(t)), \quad (5)$$

where $\mu_\theta(x_t, t)$ is the learned mean, parameterized by a neural network, and $\Sigma_\theta(t)$ is the variance, often fixed or learned. The training objective optimizes the network to minimize the difference between the true and predicted reverse distributions, typically using a simplified variational bound. During sampling, the reverse process starts from a noise sample $x_T \sim \mathcal{N}(0, I)$ and iteratively applies $p_\theta(x_{t-1}|x_t)$ to generate a sample from the target distribution. For further details on DDPM, see my previous article, TorchDiff: A PyTorch Library for Denoising Diffusion Probabilistic Models and Beyond.

3 Score-Based Generative Modeling with SDEs

Score-based generative modeling with SDEs unifies and generalizes discrete diffusion methods like SMLD and DDPM by modeling the diffusion process as a continuous-time stochastic differential equation (SDE). Instead of perturbing data with a finite number of noise scales, the SDE framework

considers a continuum of noisy distributions evolving over time $t \in [0, T]$ according to a forward SDE:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t) dt + g(t) d\mathbf{w}, \quad (6)$$

where $\mathbf{f}(\mathbf{x}, t)$ is the drift term, $g(t)$ is the diffusion coefficient, and $d\mathbf{w}$ is the increment of a standard Wiener process. This forward process progressively transforms data $\mathbf{x}(0) \sim p_0$ into noise $\mathbf{x}(T) \sim p_T$ (typically a standard normal distribution) without requiring trainable parameters.

The reverse process, which transforms noise back into data, is governed by a reverse-time SDE (Anderson, 1982):

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}}, \quad (7)$$

where $d\bar{\mathbf{w}}$ is the increment of a reverse-time Wiener process, and $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ is the score of the noisy data distribution at time t . A time-dependent neural network is trained to approximate this score, enabling the reverse SDE to be solved numerically using SDE solvers (e.g., Euler-Maruyama or stochastic Runge-Kutta methods). This process generates high-quality samples by iteratively denoising samples from p_T to approximate p_0 .

The SDE framework's key advantage is its flexibility: it generalizes discrete diffusion processes (like SMLD and DDPM) into a continuous-time formulation, allowing for adaptive noise schedules and advanced numerical methods. Figure 1 illustrates this process.

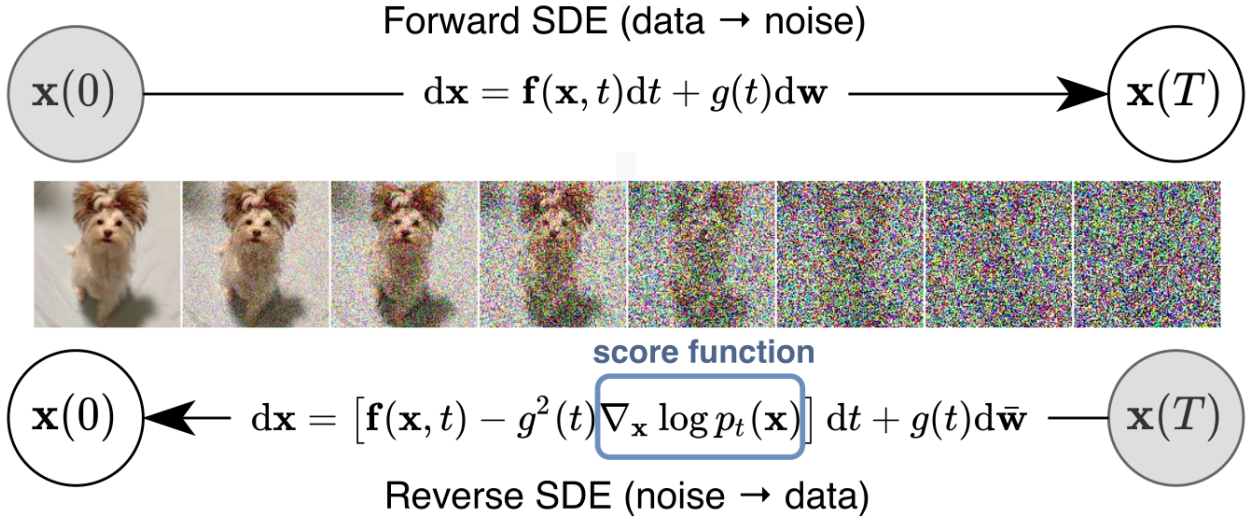


Figure 1: Illustration of the SDE framework for score-based generative modeling. The top row depicts the **forward SDE**, where clean data $\mathbf{x}(0)$ is perturbed into noise $\mathbf{x}(T)$ via the SDE $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t) dt + g(t) d\mathbf{w}$, with $\mathbf{w}$ representing a standard Wiener process. The bottom row shows the **reverse SDE**, where noise is transformed back into data using the learned score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, yielding the reverse-time SDE: $d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}}$, where $\bar{\mathbf{w}}$ is a reverse-time Wiener process. Source: Score-Based Generative Modeling through Stochastic Differential Equations.

4 Forward SDE

The forward stochastic differential equation (SDE) governs the *forward diffusion process*, where data (e.g., images or audio) is progressively corrupted by noise, transforming the original data distribution p_0 into a simple noise distribution, typically a standard normal $\mathcal{N}(0, I)$, over time $t \in [0, T]$. The forward SDE is expressed as:

$$d\mathbf{x}_t = \mathbf{f}(\mathbf{x}_t, t) dt + g(t) d\mathbf{w}_t, \quad (8)$$

where:

- $\mathbf{x}_t \in \mathbb{R}^d$: The state of the system at time t , representing the noisy data.
- $\mathbf{f}(\mathbf{x}_t, t) : \mathbb{R}^d \times [0, T] \rightarrow \mathbb{R}^d$: The drift term, governing the deterministic evolution of the system.
- $g(t) : [0, T] \rightarrow \mathbb{R}$: The diffusion coefficient, scaling the magnitude of the stochastic noise.
- $d\mathbf{w}_t$: The increment of a standard Wiener process (Brownian motion), introducing random Gaussian noise.

The forward SDE is *prescribed*, meaning it is defined without dependence on the data distribution or trainable parameters. It transforms the data from $\mathbf{x}_0 \sim p_0$ at $t = 0$ to $\mathbf{x}_T \sim \mathcal{N}(0, I)$ at $t = T$, providing a tractable prior for the reverse process.

5 Reverse-Time SDE

The reverse-time SDE governs the process of transforming the noise distribution back into the data distribution, effectively “undoing” the forward diffusion. This process leverages the mathematical framework of the Fokker-Planck equation and Girsanov’s theorem, as formalized by Anderson (1982). For the forward SDE (Equation 1), the evolution of the probability density $p_t(\mathbf{x}_t)$ is described by the Fokker-Planck equation:

$$\frac{\partial p_t(\mathbf{x}_t)}{\partial t} = -\nabla_{\mathbf{x}} \cdot [\mathbf{f}(\mathbf{x}_t, t)p_t(\mathbf{x}_t)] + \frac{1}{2}g(t)^2 \Delta p_t(\mathbf{x}_t), \quad (9)$$

where $\nabla_{\mathbf{x}} \cdot$ denotes the divergence operator, and Δ is the Laplacian.

The reverse process, running backward from $t = T$ to $t = 0$, is also a diffusion process, with its SDE derived using the time-reversed drift (Anderson, 1982). The reverse-time SDE is given by:

$$d\mathbf{x}_t = [\mathbf{f}(\mathbf{x}_t, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)] dt + g(t) d\bar{\mathbf{w}}_t, \quad (10)$$

where:

- $\mathbf{x}_t \in \mathbb{R}^d$: The state at time t , evolving backward from $\mathbf{x}_T \sim \mathcal{N}(0, I)$ at $t = T$ to $\mathbf{x}_0 \approx p_0$ at $t = 0$.
- $\mathbf{f}(\mathbf{x}_t, t)$: The same drift term as in the forward SDE.

- $g(t)$: The same diffusion coefficient as in the forward SDE.
- $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$: The *score*, defined as the gradient of the log-probability density of the noisy data distribution at time t .
- $d\bar{\mathbf{w}}_t$: The increment of a Wiener process in the reverse-time direction.

The additional term $-g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$ arises from Girsanov’s theorem, which adjusts the drift to align the reverse process with the marginal density $p_t(\mathbf{x}_t)$. The diffusion coefficient $g(t)$ remains unchanged, as the stochastic noise is symmetric in time, and $d\bar{\mathbf{w}}_t$ accounts for randomness in the backward direction. This reverse process enables sampling from the original data distribution by iteratively denoising samples from $\mathcal{N}(0, I)$.

6 Estimating the Score Function

To implement the reverse-time SDE, the score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ must be estimated, as it is typically intractable for complex data distributions. A neural network $s_{\theta}(\mathbf{x}, t)$, often a U-Net with time embeddings and attention mechanisms, is trained to approximate this score using *denoising score matching*.

The training objective is to minimize the following loss function:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{t \sim \mathcal{U}[0, T]} \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0) \sim p_0, \mathbf{x}(t) \sim p_{0t}(\cdot | \mathbf{x}(0))} \left[\|s_{\theta}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))\|_2^2 \right] \right\}, \quad (11)$$

where:

- $\lambda(t)$: A weighting function, often set to $\lambda(t) = g(t)^2$ to balance the contribution of different time steps.
- $\mathcal{U}[0, T]$: A uniform distribution over the time interval $[0, T]$.
- $p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))$: The forward transition density of the diffusion process, describing the distribution of $\mathbf{x}(t)$ given the initial data $\mathbf{x}(0)$.

With sufficient data and model capacity, the optimal score model satisfies:

$$s_{\theta}^*(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}), \quad (12)$$

closely approximating the true score function. This denoising score matching approach effectively trains the neural network to predict the score by leveraging the relationship between the noisy data $\mathbf{x}(t)$ and the clean data $\mathbf{x}(0)$, enabling efficient sampling via the reverse-time SDE.

7 Variance Exploding, Variance Preserving, and sub-VP SDEs

Score-based generative modeling leverages stochastic differential equations (SDEs) to model the gradual addition of noise to data and the reverse process of denoising to generate samples from

complex data distributions. As introduced in Score-Based Generative Modeling through Stochastic Differential Equations, three key SDEs are commonly used: the Variance Exploding (VE) SDE, the Variance Preserving (VP) SDE, and the sub-VP SDE. Each defines a distinct continuous-time diffusion process with unique characteristics for noise injection and sampling. This section explores these SDEs and their roles in score-based generative modeling.

7.1 Variance Exploding (VE) SDE

The Variance Exploding (VE) SDE is characterized by a rapidly increasing variance, driven solely by stochastic noise without a stabilizing drift term. It is defined as:

$$d\mathbf{x}_t = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}_t, \quad (13)$$

where $\mathbf{w}_t$ is a standard Wiener process (Brownian motion), and $\sigma(t)$ is a time-dependent function controlling the noise scale. The term $\frac{d[\sigma^2(t)]}{dt}$ governs the rate of variance growth. The absence of a drift term means the process is purely stochastic, leading to an “exploding” variance over time.

The marginal distribution of the state $\mathbf{x}_t$ at time t , given an initial state $\mathbf{x}_0 \sim p_0$, is:

$$\mathbf{x}_t \sim \mathcal{N}(\mathbf{x}_0, \sigma^2(t)I). \quad (14)$$

Here, the mean remains $\mathbf{x}_0$, but the variance $\sigma^2(t)$ grows significantly, often exponentially. A common noise schedule is a geometric progression:

$$\sigma(t) = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^t, \quad (15)$$

leading to the SDE:

$$d\mathbf{x}_t = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^t \sqrt{2 \log \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)} d\mathbf{w}_t. \quad (16)$$

Here, $\sigma_{\min}$ and $\sigma_{\max}$ are hyperparameters defining the initial and maximum noise scales. The VE SDE is the continuous-time analogue of Score Matching with Langevin Dynamics (SMLD), as it heavily corrupts data into a near-isotropic Gaussian distribution, facilitating robust score function learning. However, the exploding variance can challenge numerical stability, requiring careful implementation in frameworks like TorchDiff.

7.2 Variance Preserving (VP) SDE

The Variance Preserving (VP) SDE maintains bounded variance throughout the diffusion process, making it ideal for applications like Denoising Diffusion Probabilistic Models (DDPMs), which balance sampling quality and likelihood estimation. The VP SDE is defined as:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)} d\mathbf{w}_t, \quad (17)$$

where $\beta(t)$ is a time-dependent function controlling both the drift and diffusion terms. The drift term $-\frac{1}{2}\beta(t)\mathbf{x}_t$ shrinks the state toward zero, while the diffusion term $\sqrt{\beta(t)} d\mathbf{w}_t$ injects noise. This interplay ensures controlled variance growth.

The marginal distribution of $\mathbf{x}_t$ is:

$$\mathbf{x}_t \sim \mathcal{N} \left(\mathbf{x}_0 \cdot e^{-\frac{1}{2} \int_0^t \beta(s) ds}, \left(1 - e^{-\int_0^t \beta(s) ds} \right) I \right). \quad (18)$$

The mean is scaled by $e^{-\frac{1}{2} \int_0^t \beta(s) ds}$, reflecting the drift's effect, while the variance $1 - e^{-\int_0^t \beta(s) ds}$ remains bounded, approaching 1 as $t \rightarrow \infty$ for normalized data with unit variance. The variance $\Sigma(t) = \text{Cov}[\mathbf{x}_t]$ evolves according to:

$$\frac{d\Sigma(t)}{dt} = \beta(t)(I - \Sigma(t)), \quad (19)$$

with the solution:

$$\Sigma(t) = I - e^{-\int_0^t \beta(s) ds} (I - \Sigma(0)). \quad (20)$$

This ensures that the variance converges to the identity matrix I , enhancing stability. The VP SDE's bounded variance makes it suitable for precise noise control, often using linear or cosine schedules for $\beta(t)$, as implemented in frameworks like TorchDiff.

7.3 sub-VP SDE

The sub-VP SDE modifies the VP SDE to reduce the rate of variance growth, improving likelihood estimation while maintaining strong sampling performance. It is defined as:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t) \left(1 - e^{-2 \int_0^t \beta(s) ds} \right)} d\mathbf{w}_t. \quad (21)$$

The drift term matches that of the VP SDE, yielding the same mean behavior:

$$\mathbb{E}[\mathbf{x}_t] = \mathbf{x}_0 \cdot e^{-\frac{1}{2} \int_0^t \beta(s) ds}. \quad (22)$$

However, the diffusion term is scaled by $\sqrt{1 - e^{-2 \int_0^t \beta(s) ds}}$, which reduces noise injection compared to the VP SDE. The variance evolves as:

$$\Sigma_{\text{sub-VP}}(t) = \left(1 - e^{-\int_0^t \beta(s) ds} \right) I + e^{-2 \int_0^t \beta(s) ds} (\Sigma(0) - I). \quad (23)$$

This results in a tighter variance bound than the VP SDE, as $\Sigma_{\text{sub-VP}}(t) \leq \Sigma_{\text{VP}}(t)$. Both SDEs converge to a variance of I as $t \rightarrow \infty$, but the sub-VP SDE does so more slowly, enhancing training stability and likelihood estimation, particularly in probabilistic modeling tasks.

Property	VE SDE	VP SDE	sub-VP SDE
Drift term	0	$-\frac{1}{2}\beta(t)\mathbf{x}_t$	$-\frac{1}{2}\beta(t)\mathbf{x}_t$
Diffusion term	$\sqrt{\frac{d[\sigma^2(t)]}{dt}}$	$\sqrt{\beta(t)}$	$\sqrt{\beta(t) \left(1 - e^{-2 \int_0^t \beta(s) ds}\right)}$
Variance behavior	Exploding	Bounded	Bounded (tighter)
Mean behavior	$\mathbf{x}_0$	$\mathbf{x}_0 \cdot e^{-\frac{1}{2} \int_0^t \beta(s) ds}$	Same as VP
Reverse SDE used?	Yes	Yes	Yes

Table 1: Comparison of VE, VP, and sub-VP SDEs in score-based generative modeling.

7.4 Comparing VE, VP, and sub-VP SDEs

The VE, VP, and sub-VP SDEs offer distinct characteristics for score-based generative modeling, as summarized in the following table:

The VE SDE excels at heavily noising data, aligning with SMLD and enabling robust score learning, but its exploding variance requires careful numerical handling. The VP SDE, closely related to DDPMs, balances sampling quality and likelihood estimation with bounded variance, making it versatile for many applications. The sub-VP SDE refines the VP SDE by slowing variance growth, improving likelihood estimation and training stability without compromising sampling quality.

8 Probability Flow ODE in Score-Based Generative Modeling

In score-based generative modeling, samples are typically generated by solving the reverse-time stochastic differential equation (SDE), as discussed in previous sections. However, an equivalent deterministic process, known as the *Probability Flow ODE*, can be derived to produce samples with the same time-marginal distributions $p_t(\mathbf{x})$ as the reverse SDE. This deterministic formulation eliminates stochasticity, enabling the use of efficient numerical ODE solvers and exact likelihood computation.

Given the forward SDE:

$$d\mathbf{x}_t = \mathbf{f}(\mathbf{x}_t, t) dt + g(t) d\mathbf{w}_t, \quad (24)$$

the corresponding Probability Flow ODE is:

$$d\mathbf{x}_t = \left[\mathbf{f}(\mathbf{x}_t, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t) \right] dt, \quad (25)$$

where:

- $\mathbf{f}(\mathbf{x}_t, t) : \mathbb{R}^d \times [0, T] \rightarrow \mathbb{R}^d$ is the drift term from the forward SDE,
- $g(t) : [0, T] \rightarrow \mathbb{R}$ is the diffusion coefficient,

- $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$ is the score function, approximated by a neural network $s_\theta(\mathbf{x}, t)$.

The Probability Flow ODE shares the same marginal distributions $p_t(\mathbf{x})$ as the forward and reverse SDEs but evolves deterministically from $t = T$ (noise, $\mathbf{x}_T \sim \mathcal{N}(0, I)$) to $t = 0$ (data, $\mathbf{x}_0 \approx p_0$). By integrating this ODE using numerical solvers (e.g., Runge-Kutta methods), samples can be generated without the randomness inherent in SDE-based sampling, improving computational efficiency and reproducibility.

A key advantage of the Probability Flow ODE is its ability to compute exact likelihoods, akin to normalizing flows. The log-likelihood of a data point $\mathbf{x}_0$ is given by the instantaneous change-of-variables formula:

$$\log p_0(\mathbf{x}_0) = \log p_T(\mathbf{x}_T) - \int_0^T \nabla_{\mathbf{x}} \cdot \left[\mathbf{f}(\mathbf{x}_t, t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t) \right] dt, \quad (26)$$

where $p_T(\mathbf{x}_T)$ is the prior distribution (typically $\mathcal{N}(0, I)$), and $\nabla_{\mathbf{x}} \cdot$ denotes the divergence of the ODE’s vector field. This enables training and evaluating generative models with exact likelihoods, a feature particularly valuable for probabilistic modeling tasks implemented in frameworks like TorchDiff.

Feature	Reverse SDE Sampling	Probability Flow ODE
Randomness	Stochastic	Deterministic
Noise Injection	Yes (via Wiener process)	No
Solver Type	SDE solver (e.g., Euler–Maruyama)	ODE solver (e.g., Runge–Kutta)
Score Usage	In drift term	In vector field
Likelihood Computation	Intractable	Exact (via change of variables)
Latent Representation	Implicit	Explicit, invertible
Applications	High-quality sampling	Sampling, likelihood estimation, interpolation

Table 2: Comparison of sampling via reverse-time SDEs and the Probability Flow ODE in score-based generative modeling.

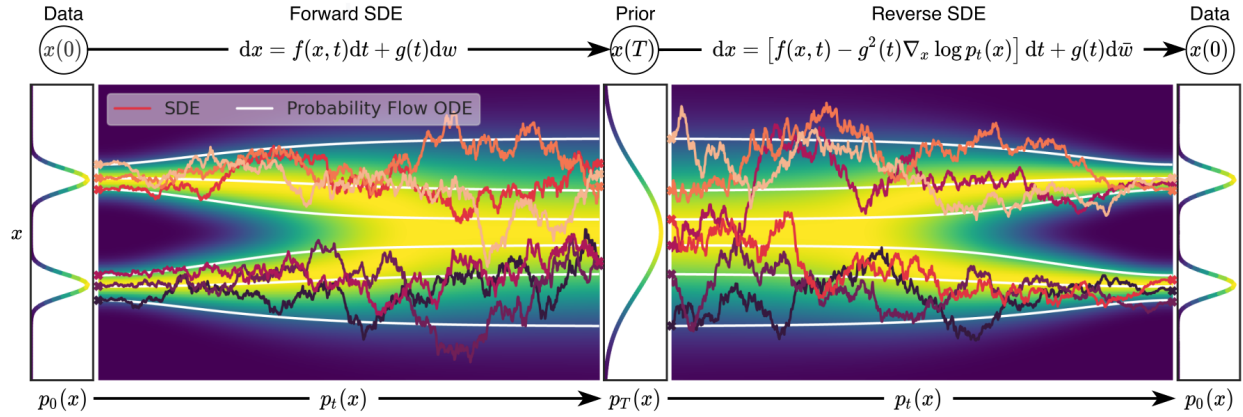


Figure 2: Visualization of stochastic and deterministic processes in score-based generative modeling. The **forward SDE** (left) transforms data from the distribution $p_0(\mathbf{x})$ to a prior $p_T(\mathbf{x})$ via $d\mathbf{x}_t = \mathbf{f}(\mathbf{x}_t, t) dt + g(t) d\mathbf{w}_t$, with sample trajectories (red lines) and evolving density $p_t(\mathbf{x})$ (background). The **reverse SDE** (right) generates data from noise using $d\mathbf{x}_t = [\mathbf{f}(\mathbf{x}_t, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)] dt + g(t) d\bar{\mathbf{w}}_t$. The **Probability Flow ODE** (white lines) follows a deterministic path with the same marginal distributions $p_t(\mathbf{x})$, enabling exact likelihood computation and efficient sampling.

9 Implementing SDEs with TorchDiff

TorchDiff is a PyTorch-based library designed to simplify the implementation of diffusion models, including Denoising Diffusion Probabilistic Models (DDPM), Denoising Diffusion Implicit Models (DDIM), and score-based generative models using stochastic differential equations (SDEs). As described in my previous article, TorchDiff: A PyTorch Library for Denoising Diffusion Probabilistic Models and Beyond, TorchDiff provides modular components for building and training diffusion models. This section demonstrates how to use TorchDiff to train and sample from a conditional SDE-based model, such as Variance Exploding (VE), Variance Preserving (VP), sub-VP, or the Probability Flow ODE, using the MNIST dataset.

9.1 Preparing the Dataset

To train an SDE-based model, we first prepare the dataset using PyTorch’s data handling utilities, as TorchDiff is built on PyTorch. The following example loads the MNIST dataset, a collection of 28x28 grayscale handwritten digit images, and applies preprocessing transformations to resize and normalize the images.

```
1 import torch
2 from torchvision import datasets, transforms
3 from torch.utils.data import DataLoader
4
5 # Define image preprocessing transformations
6 transform = transforms.Compose([
7     transforms.Resize(32),           # Resize to 32x32 pixels
8     transforms.CenterCrop(32),       # Ensure size is 32x32
```

```

9     transforms.ToTensor(),                # Convert to tensor (values in [0,
10         1])
11     transforms.Normalize((0.5,), (0.5,)) # Normalize for grayscale
12 ])
13 # Load MNIST training and validation datasets
14 train_dataset = datasets.MNIST(
15     root='./mnist/train', train=True,
16     transform=transform, download=True
17 )
18 val_dataset = datasets.MNIST(
19     root='./mnist/val', train=False,
20     transform=transform, download=True
21 )
22
23 # Create data loaders
24 train_loader = DataLoader(
25     train_dataset, batch_size=64, shuffle=True,
26     num_workers=8, pin_memory=True
27 )
28 val_loader = DataLoader(
29     val_dataset, batch_size=64, shuffle=False,
30     num_workers=8, pin_memory=True
31 )

```

Listing 1: Preparing the MNIST dataset for training.

This code prepares the MNIST dataset for training by resizing images to 32x32 pixels, normalizing pixel values, and creating data loaders for efficient batch processing.

9.2 Configuring the SDE Model

Next, we configure the components of a conditional SDE-based model, including the SDE hyperparameters, reverse diffusion process, noise predictor (a U-Net), text encoder for conditioning, optimizer, loss function, and evaluation metrics. The following example sets up a model using the Probability Flow ODE, with options to switch to VE, VP, or sub-VP SDEs.

```

1 import torch
2 import torch.nn as nn
3 from torchdiff.sde import HyperParamsSDE, ReverseSDE
4 from torchdiff.utils import TextEncoder, NoisePredictor, Metrics
5
6 # SDE hyperparameters
7 hp_sde = HyperParamsSDE(
8     num_steps=500, beta_start=1e-4, beta_end=0.02, sigma_start=1e-3,
9     sigma_end=10.0
10 )
11
12 # Reverse diffusion (e.g., ODE method)
13 rev_sde = ReverseSDE(hp_sde, method="ode") # Options: "ve", "vp", "sub_vp", "ode"
14
15 # Noise predictor (U-Net for denoising)

```

```

15 noise_pred = NoisePredictor(
16     in_channels=1, # Grayscale input
17     down_channels=[32, 64, 128, 256],
18     mid_channels=[256, 256, 256, 256],
19     up_channels=[256, 128, 64, 32],
20     time_embed_dim=256,
21     y_embed_dim=256
22 )
23
24 # Text encoder for conditioning
25 text_enc = TextEncoder(model_name="bert-base-uncased", output_dimension
26                        =256)
27
28 # Optimizer and loss function
29 optim = torch.optim.Adam(
30     list(noise_pred.parameters()) + list(text_enc.parameters()), lr=1e-4
31 )
32 loss_fn = nn.MSELoss()
33
34 # Evaluation metrics
35 metrics = Metrics(device="cuda", fid=True, ssim=True, lpips=True)

```

Listing 2: Configuring modules for training a text-conditional SDE model.

This configuration defines a flexible SDE model where the ‘method’ parameter allows easy switching between SDE types or the Probability Flow ODE. The noise predictor is a U-Net tailored for denoising, and the text encoder enables text-conditional generation.

9.3 Training and Sampling

Finally, we use TorchDiff’s ‘TrainSDE’ and ‘SampleSDE’ modules to train the model on the MNIST dataset and generate samples. The following code demonstrates the training and sampling process.

```

1 from torchdiff.sde import TrainSDE, SampleSDE
2
3 # Train the SDE model
4 trainer = TrainSDE(
5     method="ode",
6     noise_predictor=noise_pred,
7     hyper_params=hp_sde,
8     conditional_model=text_enc,
9     metrics_=metrics,
10    optimizer=optim,
11    objective=loss_fn,
12    data_loader=train_loader,
13    val_loader=val_loader,
14    max_epoch=100,
15    device="cuda",
16    store_path="sde_model.pth",
17    val_frequency=10
18 )
19 trainer() # Start training
20

```

```

21 # Generate a sample
22 sampler = SampleSDE(
23     reverse_diffusion=rev_sde,
24     noise_predictor=noise_pred,
25     image_shape=(1, 32, 32), # Grayscale, 32x32
26     conditional_model=text_enc,
27     batch_size=1,
28     in_channels=1,
29     device="cuda"
30 )
31 image = sampler(conditions=["A handwritten digit"]) # Generate one
           conditioned image

```

Listing 3: Training and sampling with a text-conditional SDE model.

The ‘TrainSDE’ module handles the training loop, optimizing the noise predictor and text encoder to learn the score function for the specified SDE or ODE. After training, the ‘SampleSDE’ module generates samples by solving the reverse diffusion process, conditioned on a text prompt.

10 Conclusion

Score-based generative modeling with SDEs offers a flexible and theoretically grounded approach to generating high-quality samples from complex data distributions. By unifying discrete diffusion models like SMLD and DDPM under a continuous-time framework, SDEs such as VE, VP, and sub-VP, along with the Probability Flow ODE, provide versatile tools for both sampling and likelihood estimation. TorchDiff simplifies the implementation of these models, enabling practitioners to train and sample from SDE-based models with ease, as demonstrated with the MNIST dataset. This article bridges theoretical insights with practical applications, empowering readers to leverage SDEs and TorchDiff for cutting-edge generative modeling tasks. For further exploration, the open-source TorchDiff library and its documentation provide a robust starting point for building custom diffusion models.

11 References

References

- [1] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://arxiv.org/abs/2011.13456>
- [2] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: <https://arxiv.org/abs/2006.11239>

- [3] Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.05600>
- [4] B. D. O. Anderson, “Reverse-time diffusion equation models,” *Stochastic Processes and their Applications*, vol. 12, no. 3, pp. 313–326, 1982. [Online]. Available: [https://doi.org/10.1016/0304-4149\(82\)90051-5](https://doi.org/10.1016/0304-4149(82)90051-5)
- [5] S. Loqman, “TorchDiff: A PyTorch Library for Denoising Diffusion Probabilistic Models and Beyond,” *Medium*, 2024. [Online]. Available: <https://medium.com/@samaniLoqman91/torchdiff-a-pytorch-library-for-denoising-diffusion-probabilistic-models-and-beyond>
- [6] S. Loqman, “TorchDiff: A PyTorch Library for Diffusion Models,” [Online]. Available: <https://github.com/LoqmanSamani/TorchDiff>